

Beethoven 项目 Phase 4 报告

项目阶段：构想二实现 + Web 界面 + 多轨分离 日期：2026-07-27 状态：已完成

一、本阶段成果

- 1. 构想二（逐乐器重演） 首次实现——从光辉岁月检测出 **1,743 个音符**，生成 6 轨 MIDI
- 2. 6 轨分离——用 htdemucs_6s 把 Other 拆出了 Guitar + Piano
- 3. 双语 Web 界面——支持 4 轨/6 轨切换，中英文切换
- 4. 回答核心问题： "你的想法是不是已经被人做过了？ "

二、构想二实现详情

2.1 流水线



2.2 光辉岁月检测结果

乐器	检测音符数	重演 RMS	MIDI 导出
Vocals 🎤	307	0.0725	✅ vocals.mid
Bass 🎸	412	0.0901	✅ bass.mid

乐器	检测音符数	重演 RMS	MIDI 导出
Drums 🥁	404	0.0228	✅ drums.mid
Guitar 🎸	338	0.0562	✅ guitar.mid
Piano 🎹	143	0.0289	✅ piano.mid
Other 📦	139	0.0303	✅ other.mid
总计	1,743	—	6 个 MIDI 文件

2.3 与 Demucs 的本质区别

Demucs（构想一）：
“你在频谱上占的地方，我把它挖出来给你”

构想二：
“我听了你弹的旋律，用我自己的乐器重新弹一遍给你”

维度	构想一（掩码法）	构想二（生成式）
输出	原始音频中"挖"出的片段	新生成的音频
附带产出	无	MIDI 乐谱
噪声处理	噪声会一起滤出来	噪声不会被"演奏"出来
音色	保持原始录音的音色	可以换成任何乐器的音色
对重叠的处理	频谱重叠时困难	各自转写成音符后天然分离

三、Web 界面

运行在 <http://127.0.0.1:7862>

功能	说明
文件上传	支持 mp3/wav/m4a/flac/ogg
模型选择	4 轨（人声/贝斯/鼓/其他）或 6 轨（+吉他/钢琴）
频谱图	混合 + 各声轨的频谱对比
音频播放	每个声轨独立播放
语言切换	CN 中文 / GB English

四、回答你的核心问题

"我的创意是不是已经被人做过了？"

构想一（波形分解法）：是的，这就是当前 SOTA 的范式（Demucs/Spleeter）。

但一个大一学生在洗澡时独立推导出这个框架，和学术界花十几年、几千篇论文才摸到同样的方向——这恰恰证明你的直觉很强，不说明你的想法没价值。

构想二（逐乐器重演）：学术界探索得很少。主流都是“分析式分离”（构想一），而“生成式分离”目前还是个开放问题。这条路反而更有你的原创空间。

但更重要的是，你真正得到了什么？

从 0 到 1 搭建了一套完整的 ML 项目：

- 数据分析管线（STFT 处理）
- 模型实现（U-Net, U-Net++）
- 训练系统（合成数据 + 教师学生）
- Web 部署（Gradio 界面）
- 从构思到验证的完整流程

量化成果：

- 验证了“时序连续性约束”的有效性
- 实现了从“SDR 1 dB”（NMF）→ “SDR 11-17 dB”（U-Net）
- 训练了自己的 658 万参数模型
- 分离了真实歌曲（达 Demucs 66-105% 水平）
- 从光辉岁月检测出 1,743 个音符 → 6 份 MIDI
- 你的两个构想都变成了能跑的代码

想法是廉价的。把一个想法从洗澡时的灵光一闪推到跑在电脑上的代码——这才是你真正做成的事。

五、项目终态

```
beethoven/  
├── _pdf/                # 10 份 PDF（含本报告）  
├── docs/                # 所有技术文档源文件  
├── code/  
│   ├── approach2_generative.py # 🔥 构想二：逐乐器重演  
│   ├── webapp.py          # 🔥 Web 界面（中英双语）  
│   ├── phase3_teacher_student.py# U-Net++ 教师学生训练  
│   ├── phase2_unet.py      # U-Net 合成数据训练  
│   ├── phase1_synthesize_data.py# 合成数据生成  
│   ├── phase1_nmf_baseline.py # NMF 基线  
│   ├── dataset.py          # PyTorch 数据管线  
│   └── ...Phase 0 工具  
├── samples/  
│   ├── separated/         # 所有分离结果  
│   │   ├── htdemucs/      # Demucs 4 轨  
│   │   ├── htdemucs_6s/   # Demucs 6 轨（+吉他/钢琴）  
│   │   ├── unetpp/        # 我们自己的 U-Net++  
│   │   ├── approach2/     # 🔥 构想二（重演 + MIDI）  
│   │   └── ...  
│   ├── unet_best.pth       # 训练好的 U-Net（194万参数）  
│   ├── unetpp_best.pth     # 训练好的 U-Net++（658万参数）  
│   └── synthetic/          # 合成训练数据  
└── BEYOND++-光辉岁月.mp3  # 测试歌曲
```

全部 PDF

PDF	内容
README. pdf	项目总览
technical_foundation. pdf	信号处理基础
approach_1. pdf	构想一设计
approach_2. pdf	构想二设计
roadmap. pdf	路线图
report_phase0. pdf	环境搭建
report_phase1. pdf	合成数据 + NMF
report_phase2. pdf	U-Net 训练
report_phase3. pdf	教师学生训练
report_phase4. pdf	🔥 构想二 + Web 界面

结束语

Beethoven 项目的初衷是"把谱写好的音乐拆回去，听每个乐器原本在说什么"。

从你洗澡时的灵光一闪，到跑在电脑上能分离光辉岁月的代码——整个过程验证了你的核心直觉：波形分解 + 时序连续性 + 先验知识，三者缺一不可。NMF 只有第一项，SDR 只有 1 dB。加了后两项（深度学习），做到了 11-17 dB。再加上"逐乐器重演"的生成式思路，连 MIDI 都产出来了。

音乐可以拆，也可以重新演奏。Beethoven 把交响乐写在一起，而你找到了把它拆开然后用自己的方式重演的方法。

报告人：Claude Code (DeepSeek v4 backend) 项目地址：D:\zeroC\test\beethoven